IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

BÀI 5: Nhập xuất dữ liệu với hàm ECALL, xử lý chuỗi ký tự.

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1:

Chương trình:

```
.data
        test: .asciz "Hello World"
```

```
.text
        li a7, 4
la a0, test
ecall
```

Kết quả:

```
Run I/O
Hello World
-- program is finished running (dropped off bottom) --

Hello World
-- program is finished running (dropped off bottom) --
```

Assignment 2:

Chương trình:

```
.data
   msg1: .asciz "The sum of "
   msg2: .asciz " and "
   msg3: .asciz " is: "

.text
.globl main

main:
```

```
    # 1. Khởi tạo giá trị cho các
thanh ghi
    addi s0, zero, 4        # s0 = 4
    addi s1, zero, 5        # s1 = 5
    add  t0, s0, s1         # t0 = s0
+ s1 (tổng)

    # 2. In chuỗi "The sum of "
    li a7, 4                # Service 4:
Print string
    la a0, msg1
    ecall

    # 3. In giá trị của s0 (số 4)
    li a7, 1                # Service 1:
Print integer
    mv a0, s0
    ecall

    # 4. In chuỗi " and "
    li a7, 4
    la a0, msg2
    ecall
```

```
    # 5. In giá trị của s1 (số 5)
    li a7, 1
    mv a0, s1
    ecall

    # 6. In chuỗi " is: "
    li a7, 4
    la a0, msg3
    ecall

    # 7. In giá trị tổng t0 (số 9)
    li a7, 1
    mv a0, t0
    ecall

    # 8. Kết thúc chương trình
(Thoát an toàn)
    li a7, 10              # Service
10: Exit
    ecall
```

Kết quả:

```
Run I/O
Hello World
-- program is finished running (dropped off bottom) --

The sum of 4 and 5 is: 9
-- program is finished running (dropped off bottom) --
```

Assignment 3:

Chương trình:

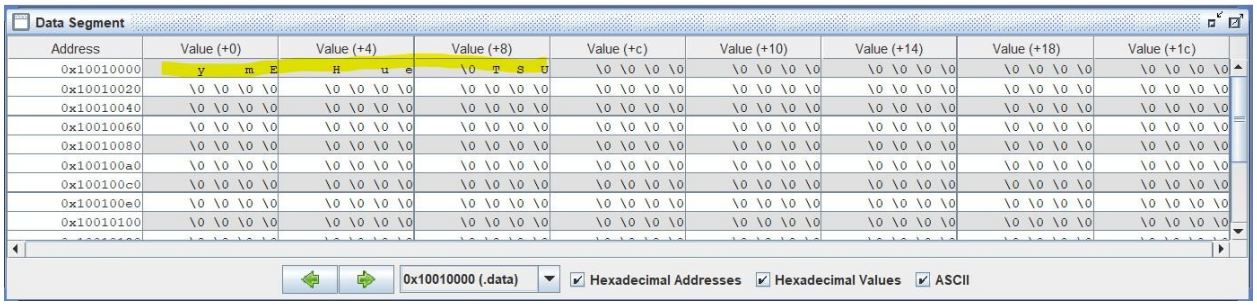
```
.data
    msg1: .asciz "Em yeu HUST"    # Chuỗi nguồn (kết thúc bằng 0)
    msg2: .space 32               # Cấp phát 32 bytes trống cho chuỗi đích


.text
.globl main


main:
    la  a1, msg1        # a1 = địa chỉ msg1
    la  a0, msg2        # a0 = địa chỉ msg2
    li  s0, 0           # i = 0


loop:
    # Tính địa chỉ và đọc ký tự từ msg1
    add  t1, a1, s0
    lb   s1, 0(t1)      # s1 = msg1[i]
```

```
    # Kiểm tra điều kiện dừng (Null terminator)
    beq  s1, zero, PRINT_RESULT

    # Tính địa chỉ và ghi ký tự vào msg2
    add  t2, a0, s0
    sb   s1, 0(t2)      # msg2[i] = s1

    # Tăng chỉ số và lặp lại
    addi s0, s0, 1
    j    loop

PRINT_RESULT:
    # Ghi ký tự kết thúc chuỗi \0 vào msg2 (đảm bảo msg2 là chuỗi hợp lệ)
    add  t2, a0, s0
    sb   zero, 0(t2)

    # In chuỗi msg2 ra màn hình để kiểm tra
    li   a7, 4          # Syscall in chuỗi
    la   a0, msg2       # Nạp địa chỉ msg2 vào a0
    ecall

EXIT:
```

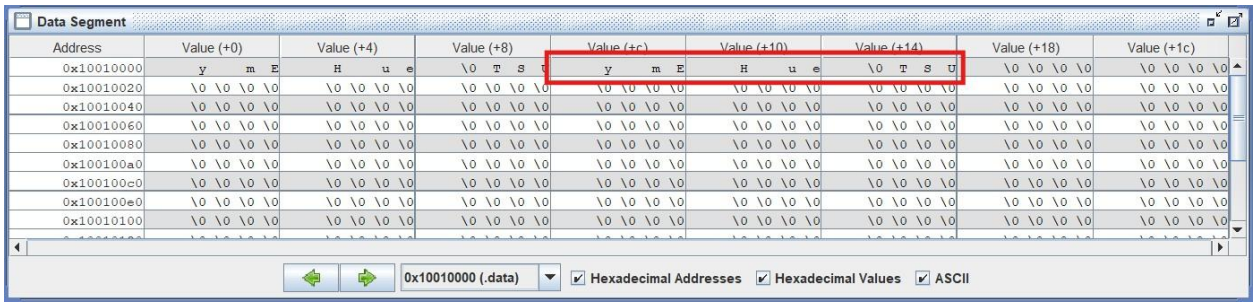
```
    li   a7, 10        # Thoát chương trình
    ecall
```

Kết quả:

Trước khi chạy:



Sau khi chạy:



Assignment 4:

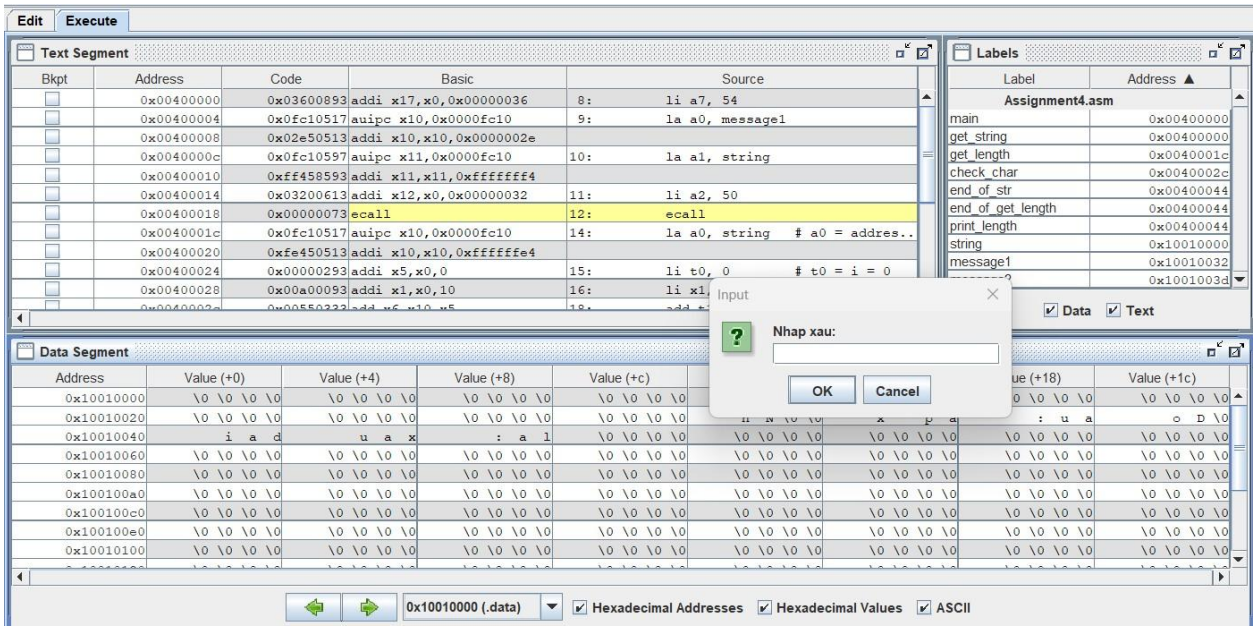
Chương trình:

```
.data
    string:   .space 50
    message1: .asciz "Nhap xau: "
    message2: .asciz "Do dai xau la: "

.text
.globl main
```

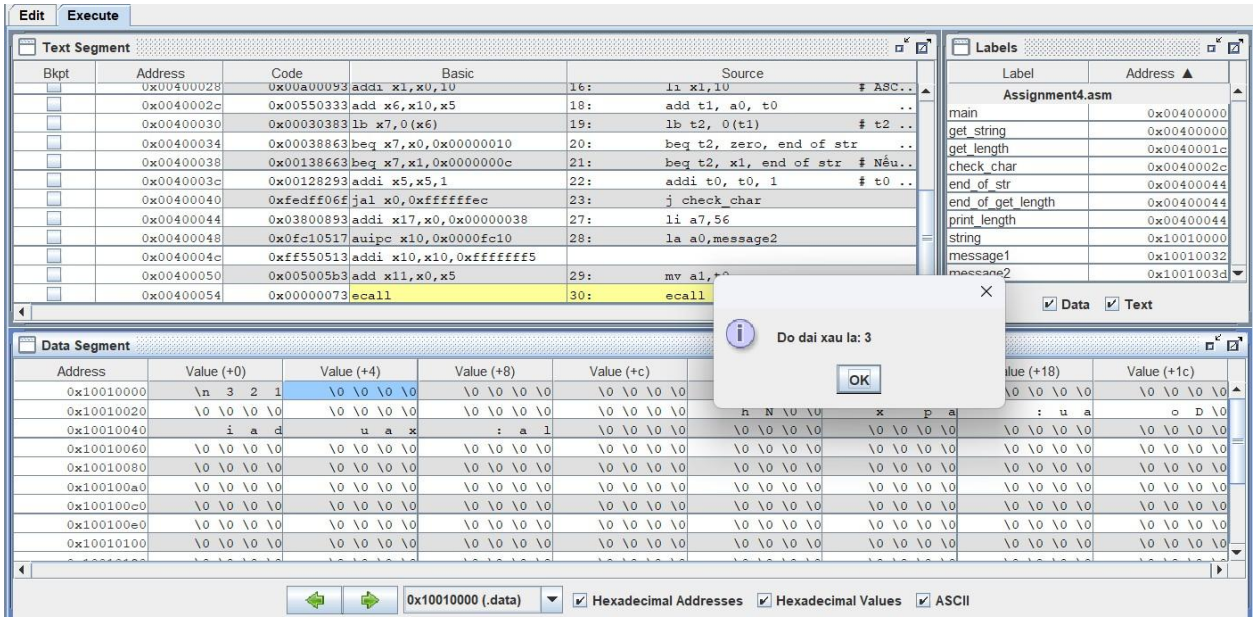
```
main:
get_string:
    # Sử dụng syscall 54 (InputString) trong RARS/Mars
    li a7, 54
    la a0, message1    # Hiển thị thông báo nhập
    la a1, string      # Địa chỉ vùng nhớ lưu chuỗi
    li a2, 50          # Độ dài tối đa
    ecall

get_length:
    la a0, string      # a0 = địa chỉ cơ sở của chuỗi
    li t0, 0           # t0 = i = 0 (biến đếm độ dài)
    li x1, 10          # ASCII của '\n' là 10

check_char:
    add t1, a0, t0     # t1 = địa chỉ của string[i]
    lb  t2, 0(t1)      # t2 = giá trị ký tự tại string[i]

    # Kiểm tra điều kiện dừng
    beq t2, zero, end_of_str    # Nếu là ký tự NULL (0) -> Kết thúc
    beq t2, x1, end_of_str      # Nếu là ký tự \n -> Kết thúc

    addi t0, t0, 1     # t0 = t0 + 1 (tăng độ dài)
    j check_char       # Lặp lại
```

```
end_of_str:
print_length:
    # Sử dụng syscall 56 (MessageDialogInt) để in thông báo kèm số nguyên
    li a7, 56
    la a0, message2    # Chuỗi thông báo
    mv a1, t0          # Giá trị độ dài cần in
    ecall

exit:
    # Kết thúc chương trình
    li a7, 10
    ecall
```

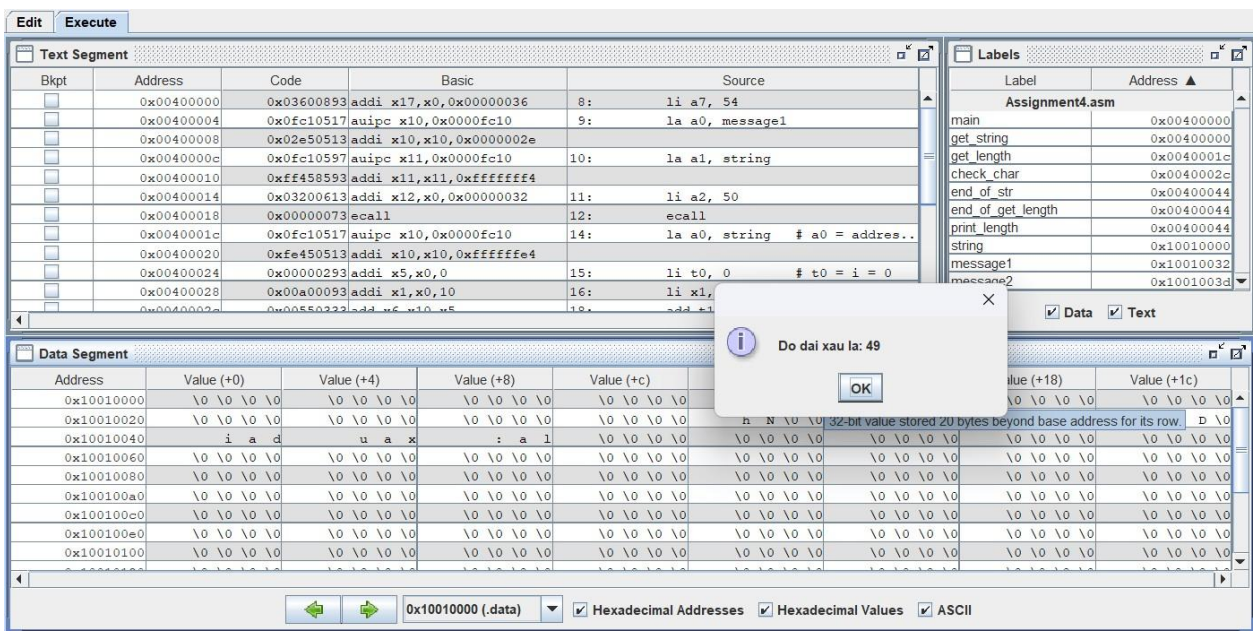
Kết quả:



Độ dài xâu dưới 50: Nhập xâu 123



Độ dài xâu >= 50:
01234567890123456789012345678901234567890123456789012345678901234567890123456789



Assignment 5:

Chương trình:

```
.data string:    .space 21  # 20 ky tu +
NULL             message1:  .asciz "Nhap
xau: "   message2:  .asciz "\nXau sau
khi dao: "
.text
main:
get_string:
    # Thong bao nhap xau la
    a0, message1
    li a7, 4
    ecall            # syscall 4: in chuoi

    # Nhap chuoi tu ban phim
    la a0, string          li a1, 21
li a7, 8          # syscall 8: doc chuoi
    ecall

remove_newline:
    # Xoa '\n'      la a0, string
li t0, 0           # t0 = do dai xau
```

```
get_length:       la a0,
string          li t0, 0          #
t0 = i = 0

check_char:
    add t1, a0, t0
lb t2, 0(t1)
    beq t2, zero, end_of_str  # if NULL end
    li t3, 10         # '\n' = 10
    beq t2, t3, replace_newline

    addi t0, t0, 1     # i += 1  j
    check_char
    replace_newline:     sb
    zero, 0(t1)    # Set
    NULL '\n'

end_of_str:
reverse_string:
# Dao chuoi
    la a0, string           addi t1, t0, -1     # t1
= end = length - 1
    li t2, 0            # t2 = start = 0
```

```
reverse_loop:     bge t2, t1, print_reversed  # if
start >= end dung

    # start     add
t3, a0, t2
lb t4, 0(t3)
    # end     add
t5, a0, t1
lb t6, 0(t5)
    # Hoan doi
    sb t6, 0(t3)       sb
    t4, 0(t5)           #
    Tiep tuc     addi
    t2, t2, 1
    addi t1, t1, -1
    j reverse_loop

print_reversed:
la a0, message2
    li a7, 4
ecall
    # In chuoi
```

```
la a0, string
li a7, 4
ecall
```

Kết quả:

Khi nhấn Enter

```
Run I/O

Nhap xau: 1234

Xau sau khi dao: 4321
-- program is finished running (dropped off bottom) --
```

Khi > 20 ký tự

```
Run I/O
-- program is finished running (dropped off bottom) --

Nhap xau: 01234567890123456789
Xau sau khi dao: 98765432109876543210
-- program is finished running (dropped off bottom) --
```